

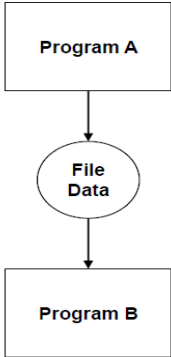
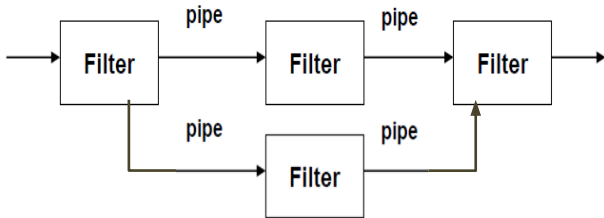
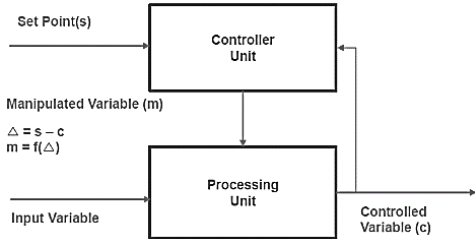


Associate Architect 양성 과정

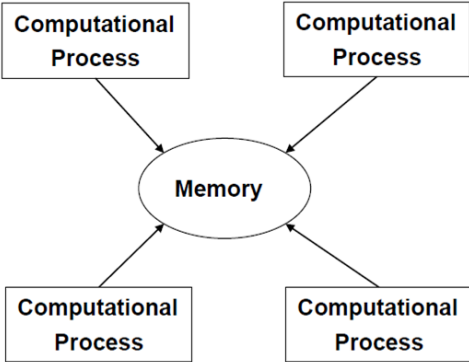
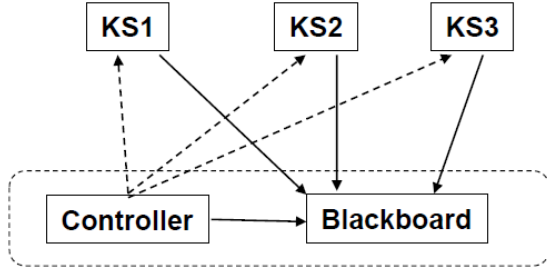
ARCHITECTURE DESIGN

Architecture Styles 요약

1. Data Flow Architecture

특징	입력 데이터를 연속해서 변환 (명확한 Data Flow)		
Style	Batch Sequential	Pipe & Filter	Process Control
구조	 <pre> graph TD A[Program A] --> B((File Data)) B --> C[Program B] </pre> • 사례: 컴파일러 시스템	 <pre> graph LR In(()) --> F1[Filter] F1 -- pipe --> F2[Filter] F1 -- pipe --> F3[Filter] F2 -- pipe --> F4[Filter] F3 -- pipe --> F4 F4 --> Out(()) </pre> • 사례: 멀티미디어 스트리머(Gstreamer), 5G 기지국 패킷 전송 Scheduler	 <pre> graph TD SP[Set Point(s)] --> CU[Controller Unit] CU -- "Manipulated Variable (m)" --> PU[Processing Unit] IV[Input Variable] --> PU PU -- "Controlled Variable (c)" --> CU CU --> PU </pre> $\Delta = s - c$ $m = f(\Delta)$ • 사례: 스마트 홈 온도 조절 시스템
차이	프로그램의 순차적 실행, 보통 파일 형태 I/O	필터의 병렬 실행 O, Pipe 입력 O → 각 필터 실행 O(점진적 실행), loop 구조 허용	피드백 루프 구조, 제어 변수 기반으로 출력을 원하는 값으로 유지

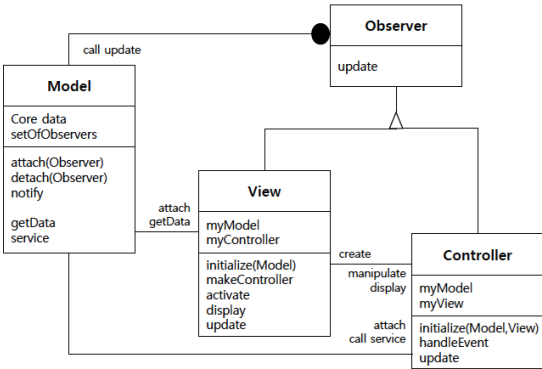
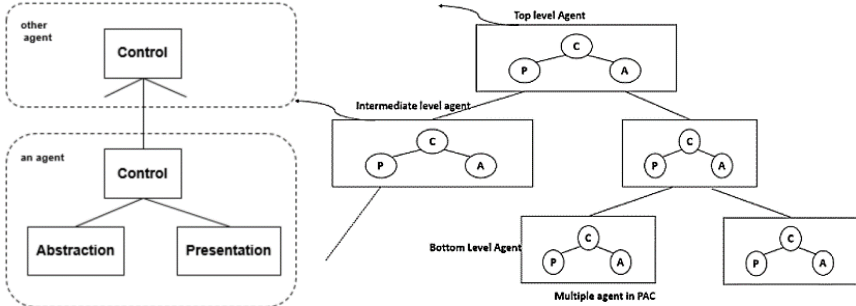
2. Data Centered Architecture

특징	중앙화 된 하나의 Data Store와 이를 공유하는 다수의 Client 구조 (Client 간 Data Store 를 통해 간접 통신)	
Style	Shared Repository	Blackboard
구조	 <pre>graph TD; CP1[Computational Process] --> Mem((Memory)); CP2[Computational Process] --> Mem; CP3[Computational Process] --> Mem; CP4[Computational Process] --> Mem;</pre> • 사례: DBMS, VCS	 <pre>graph TD; KS1[KS1] --> BB[Blackboard]; KS2[KS2] --> BB; KS3[KS3] --> BB; C[Controller] --> BB; BB -.-> KS1; BB -.-> KS2; BB -.-> KS3;</pre> • 사례: 지식 기반 인공지능 시스템, 음성 인식 비서
차이	Data Store 가 Passive, Data Client 가 Active(제어권)	Data Store 가 Active(제어권), Data Client 가 Passive

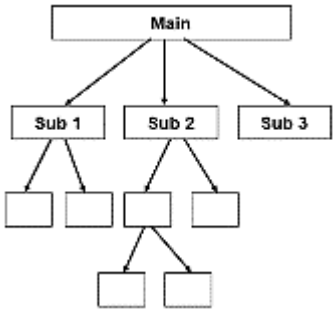
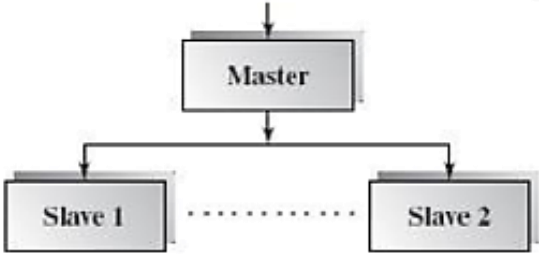
3. Implicit Invocation Architecture

특징	Receiver 컴포넌트 간 이벤트 기반 간접 호출 (loosely coupled 통신 * 1:1, 1:N, N:1, N:N 관계)	
Style	Non-Buffered Event-Based	Buffered Message-Based
구조	• 사례: 안드로이드 앱, JS 웹 브라우저	• 사례: Apache Kafka(Topic - 대용량 실시간 로그, 이벤트 스트림)
차이	중간에 버퍼 X → 동기화된 처리, Source 가 Listener 를 알고 있음 Ex: Observer Pattern	중간에 버퍼(큐) O → 비동기 처리 가능, Producer 가 Consumer 를 모름 Ex: Publisher-Subscriber Pattern

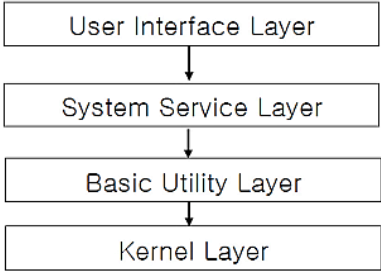
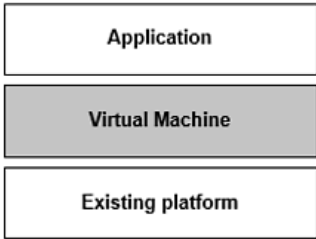
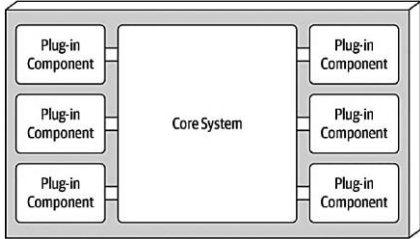
4. Interaction Oriented Architecture

특징	User Interaction 용 컴포넌트를 분리 (Interactive 한 Application 에 적합) ① 데이터 관리 & 비즈니스 로직 ② 실행 흐름 제어 ③ User Input/Output	
Style	MVC	PAC
구조	 <pre> classDiagram class Model { Core data setOfObservers attach(Observer) detach(Observer) notify getData service } class Observer { update } class View { myModel myController initialize(Model) makeController activate display update } class Controller { myModel myView initialize(Model, View) handleEvent update } Model --> Observer : call update Observer --> Model : update View -- > Observer View --> Model : attach, getData View --> Controller : create, manipulate, display Controller --> View : attach, call service Controller --> Model : initialize </pre> • 사례: SPRING Framework	 <pre> graph TD TopLevelAgent[Top level Agent P-C-A] --> IntermediateLevelAgent[Intermediate level agent P-C-A] TopLevelAgent --> BottomLevelAgent[Bottom Level Agent P-C-A] IntermediateLevelAgent --> BottomLevelAgent BottomLevelAgent --> MultipleAgent[Multiple agent in PAC] subgraph "other agent" Control1[Control] end subgraph "an agent" Control2[Control] Abstraction[Abstraction] Presentation[Presentation] end Control1 --> Control2 Control2 --> Abstraction Control2 --> Presentation </pre> • 사례: GUI 시스템, 공장 자동화 시스템(대시보드 - 공장라인 - 제어 장비)
차이	Controller 가 Model - View 간 Flow 관장, 데이터에 동기화된 View, Push 방식 Model or Pull 방식 View	Agent(P+A+C)들로 구성된 Tree 구조, Control 을 통해서만 통신, 기능성을 계층적 분해 후 할당

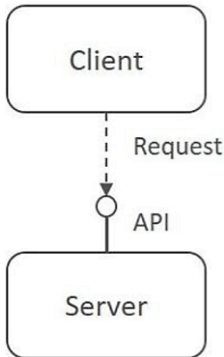
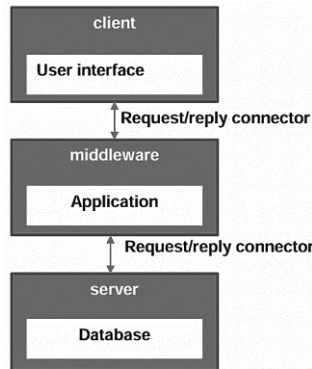
5. Hierarchical Architecture (1/2)

특징	시스템을 여러 Level 의 모듈로 분해한 계층적 구조 ① 서비스 제공: 하위 모듈 → 상위 모듈 ② 서비스 호출: 하위 모듈 ← 상위 모듈	
Style	Main-Subroutine	Master-Slave
구조	 <pre>graph TD; Main[Main] --> Sub1[Sub 1]; Main --> Sub2[Sub 2]; Main --> Sub3[Sub 3]; Sub1 --> S1_1[]; Sub1 --> S1_2[]; Sub2 --> S2_1[]; Sub2 --> S2_2[]; Sub2 --> S2_3[]; S2_3 --> S2_3_1[]; S2_3 --> S2_3_2[];</pre> • 사례: Non-OS 임베디드 기기(냉장고, 세탁기)	 <pre>graph TD; In[] --> Master[Master]; Master --> Slave1[Slave 1]; Master --> Slave2[Slave 2];</pre> • 사례: MySQL, Jenkins(CI/CD - 스케줄링/작업 분배)
차이	기능에 따라 Subroutine 으로 분해, 상위 모듈에 의해 제어 & 하위 모듈을 제어, 하나의 Control Thread	여러 Slave 가 복제된 서비스 제공, Master 가 Slave 를 제어 및 결과 선택, Slave 병렬 실행 O

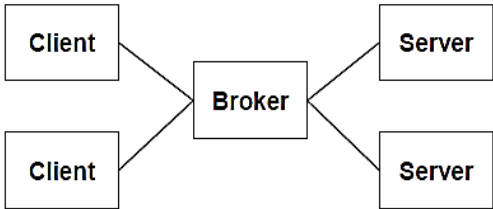
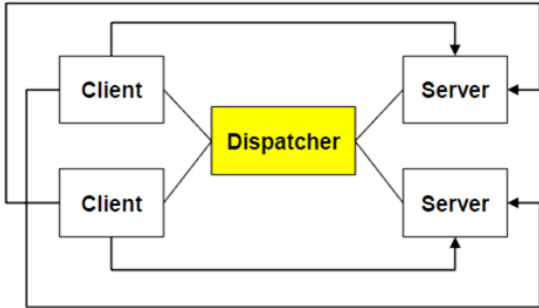
5. Hierarchical Architecture (2/2)

특징	시스템을 여러 Level 의 모듈로 분해한 계층적 구조 ① 서비스 제공: 하위 모듈 → 상위 모듈 ② 서비스 호출: 하위 모듈 ← 상위 모듈		
Style	Layered	Virtual Machine	Micro Kernel
구조		 • 사례: JVM, .NET CLR, VMWare/VirtualBox	 • 사례: Eclipse(IDE), macOS/iOS (XNU 커널)
차이	하위 Layer 만 사용 가능, Layer 간 circular 구조 허용 X	기 시스템 위에 Layer 구축, 세부사항을 감추고 추상화된 연산/속성	최소한의 Core 기능(kernel) + 추가/개선 기능(plugin)

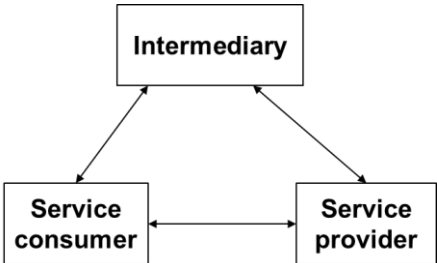
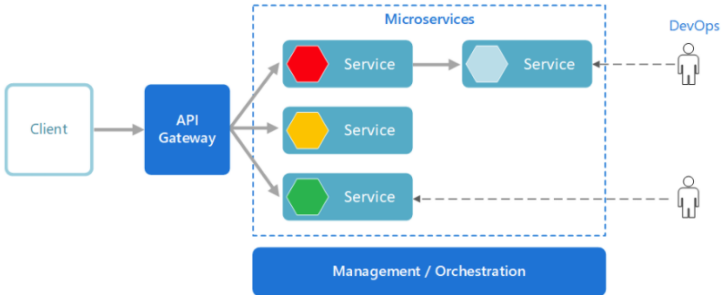
6. Distributed Architecture (1/3)

특징	여러 분산된 컴퓨팅 노드들로 시스템 구성 (네트워크 기반 통신)	
Style	Client-Server(2-tiers)	Multi-tiers(3-tiers)
구조	 <pre> graph TD Client[Client] -.-> Request API((API)) API --- Server[Server] </pre> • 사례: 웹 브라우저-서버, FTP 클라이언트-서버, 온라인 게임	 <pre> graph TD subgraph client UI[User interface] end subgraph middleware App[Application] end subgraph server DB[(Database)] end UI <--> Request/reply connector App App <--> Request/reply connector DB </pre> • 사례: 대규모 전자 상거래 시스템, 인터넷 뱅킹 시스템
차이	Client(Request 송신) + Server(Request 수신/처리, Response), Client - Server 각 컴퓨팅 리소스에 따라 역할 분배/조정 가능	책임을 구분하고 물리적으로 분해(tier)된 구조에 할당, MW 에 Load Balancing, Security 등의 추가 역할 부여 가능

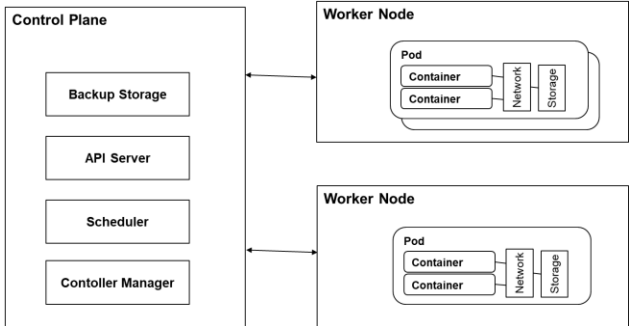
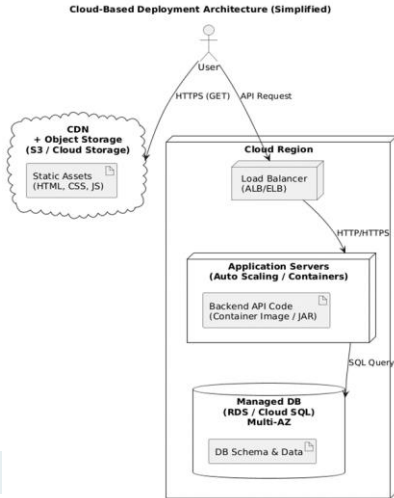
6. Distributed Architecture (2/3)

특징	여러 분산된 컴퓨팅 노드들로 시스템 구성 (네트워크 기반 통신)	
Style	Broker	Dispatcher
구조	 • 사례: IoT MQTT 프로토콜, 리눅스 데스크톱 D-Bus IPC	 • 사례: DNS 서버, Load Balancer, gRPC
차이	Client - Server 사이에서 통신을 조정/중재 (Client - Server 간 decoupled) Broker 는 Request forwarding & Translation	Dispatcher 가 Client - Server 간 연결, 통신 채널 구축 (통신 흐름 중재 X) (Client - Server 간 직접 통신)

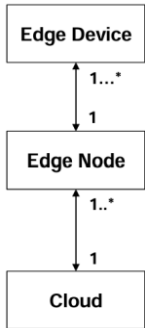
6. Distributed Architecture (3/3)

특징	여러 분산된 컴퓨팅 노드들로 시스템 구성 (네트워크 기반 통신)	
Style	Service Oriented	Microservice
구조	 • 사례: 온라인쇼핑몰 주문시스템, 물류 및 공급망 관리 시스템	 • 사례: Netflix(인증/스트리밍/추천/결제/... svc), Amazon, 배달의 민족
차이	Intermediary가 Service 공급자와 사용자간 메시지를 변환하고 Routing하며 필요에 따라 상호 작용을 중재하거나 조정하는 역할 수행 각 Service는 Business적으로 독립적이며 자체적으로 완벽한 역할 수행 가능.	독립적 개발/배포되는 작은 비즈니스 서비스들의 조합, 다양한 Client 에 API Gateway 라는 단일진입점 제공, 서비스들의 deploy, rebalancing, failure 관리 서비스 별로 독립적인 Data Store 를 보유/관리 • SOA는 Enterprise단위, Microservice는 Application단위

7. Deployment Architecture (1/2)

특징	시스템이 실제로 어디에서 어떻게 실행되는지 정의하는 구조 설계 (시스템 실행 환경, S/W Artifact, 노드간 통신 정보)	
Style	Container-Based	Cloud-Based
구조	 사례: Kubernetes, Amazon ECS, Docker	Cloud-Based Deployment Architecture (Simplified)  사례: AWS, Azure, GCP
차이	애플리케이션을 컨테이너로 패키징하여 배포, 확장, 운영을 수행하며, 각 컨테이너는 작고 가벼운 실행 환경으로 서로 완전히 격리된 공간에서 실행.	Cloud 환경을 활용해 소프트웨어를 만들고 배포하는 방법 정의. Cloud가 제공하는 특성인 탄력성, 확장성, 고가용성을 최대한 이용하여 소프트웨어를 Cloud 노드에 배치하는 방법을 설계.

7. Deployment Architecture (2/2)

특징	시스템이 실제로 어디에서 어떻게 실행되는지 정의하는 구조 설계 (시스템 실행 환경, S/W Artifact, 노드간 통신 정보)	
Style	Edge-based	
구조	 <pre>graph TD; ED[Edge Device] <--> "1...*" / "1" EN[Edge Node]; EN <--> "1..*" / "1" C[Cloud];</pre> • 사례: AWS 기반 IoT 플랫폼, 자율주행 자동차, 스마트 팩토리 (분석 센터 - 공장 별 제어 서버 - 카메라/로봇팔)	
차이	Data Source 와 가까운 곳에서 연산 처리(분산 컴퓨팅), 주요 계산은 중앙 서버에서 클라우드 컴퓨팅 → 네트워크 트래픽 문제 극복	

Thank You